# UT2. PROGRAMACIÓN MULTIHILO

## ACTIVIDAD 3

## Estados de un hilo en Java

**Título**: Observación de los estados de ejecución de un hilo en Java.

**Descripción**:

Vas a crear dos hilos y a observar cómo cambia su estado a lo largo de su ciclo de vida. Para consultar el estado de un hilo se usa el método getState() de la clase Thread, y para esperar a que un hilo termine se usa join().

**Objetivos**:

- Identificar los estados de un hilo en Java (**NEW, RUNNABLE, TIMED_WAITING, TERMINATED…**).
- Comprender cuándo y por qué cambia el estado de un hilo.
- Usar getState() para consultar el estado y join() para esperar a que un hilo termine.

**Instrucciones:**

1. **Clase MiTarea**, que implemente Runnable:
   - Su constructor recibe un array de String con los elementos que mostrará y la pausa en milisegundos entre un elemento y el siguiente.
     - MiTarea tiene dos atributos:
       - elementos (String[]): lo que el hilo mostrará por consola, uno a uno.
       - pausa (long): los milisegundos que espera el hilo después de mostrar cada elemento.
     - Ambos se reciben en el constructor:

```
public MiTarea(String[] elementos, long pausa)
```

     *Por ejemplo, new MiTarea(new String[]{"1", "2", "3", "4", "5"}, 300) crea una tarea que muestra del 1 al 5 y espera 300 ms después de cada número.*
   - En run(), recorre el array y, por cada elemento, muestra el nombre del hilo y el elemento, y después haz una pausa con Thread.sleep(). Este método lanza InterruptedException, así que debes capturarla con try/catch.
2. **Clase principal EstadosHilosDemo** (en el mismo archivo EstadosHilosDemo.java):
   - Muestra tu nombre al principio.
   - Crea dos hilos con nombre, pasándoles un objeto MiTarea:
     - "Numeros": muestra del 1 al 5 con una pausa de 300 ms.
     - "Letras": muestra de la A a la E con una pausa de 500 ms.
   - Muestra el estado de los dos hilos con getState() en estos 5 momentos:

1. Antes de llamar a start().
2. Justo después de llamar a start().
3. Tras esperar 100 ms con Thread.sleep(100).
4. Después de numeros.join(), cuando ya ha terminado el primer hilo.
5. Después de letras.join(), cuando ya han terminado los dos.

- Ejecuta el programa varias veces.

**Ten en cuenta:** en el momento 2 el estado puede cambiar de una ejecución a otra. Lo decide el planificador, no es un error.

**Preguntas** (responde en 2 o 3 líneas cada una):

1. **¿Por qué un hilo está en TIMED_WAITING y no en RUNNABLE durante la mayor parte de su ejecución?**

   Un hilo entra en estado TIMED_WAITING únicamente cuando realiza una pausa temporal con límite de tiempo. Durante la mayor parte de su ejecución activa y de procesamiento se encuentra en estado RUNNABLE.

2. **En el momento 4, ¿por qué los dos hilos tienen estados distintos?**

   Los hilos presentan estados diferentes porque el planificador del sistema operativo los gestiona y alterna de manera independiente. Mientras un hilo puede estar activo en la CPU (RUNNABLE), otro puede encontrarse esperando o durmiendo

3. **Los estados BLOCKED y WAITING no aparecen en tu programa. Explica en qué situación entraría un hilo en cada uno de ellos.**

   BLOCKED: Un hilo entra en este estado cuando intenta acceder a una sección crítica protegida por synchronized y esta se encuentra ocupada por otro hilo.

WAITING: Ocurre cuando un hilo se suspende indefinidamente al invocar métodos como wait() sin límite de tiempo o al esperar a que otro hilo concluya mediante join()

# Rúbrica:

| | |
|---|---|
| **0** | No se cumple ninguno de los requisitos mínimos. |
| **3** | El programa funciona, pero muestra menos de 3 de los 5 momentos. |
| **5** | Muestra los 5 momentos, pero falta join() o las capturas. |
| **7** | Programa y capturas correctos. Las preguntas están incompletas o tienen errores. |
| **9** | Todo correcto: código, capturas y preguntas razonadas. |

| | |
|---|---|
| **10** | Todo correcto y, además, incluye una mejora que aporta valor al programa. |

## Entrega:

i. Incluye aquí el código fuente completo de EstadosHilosDemo.java (clases MiTarea y EstadosHilosDemo):

```
/**
 * @author Juan Gabriel Galarza claros
 */

class MiTarea implements Runnable{
   // Declaración de atributos
   private String[] elementos;
   private long pausa;

   // Creamos el contructor
   public MiTarea(String[] elementos, long pausa) {
      this.elementos = elementos;
      this.pausa = pausa;
   }

   // Metodo Runnable
   @Override
   public void run() {
      // Muestra el nombre
      String nombre = Thread.currentThread().getName();

         // Recorresr los elementos del array
         for (String elemento : elementos) {
            System.out.println(nombre + "; " + elemento);
            try {
               Thread.sleep(pausa);
            } catch (InterruptedException e) {
               System.out.println(nombre + " ha sido interrumpido.");
               return;
            }
         }
   }
}

public class EstasdosHilosDemo {
   // Hio principal
   public static void main(String[] args) throws InterruptedException{
      System.out.println("Nombre y apellidos: Juan Gabriel Galarza Claros");

      MiTarea tareaNumeros = new MiTarea(new String[]{"1","2","3","4","5"}, 300);
      MiTarea tareaLetras = new MiTarea(new String[]{"A","B","C","D","E"}, 500);

      Thread hiloNumeros = new Thread(tareaNumeros, "Numeros");
```

```
        Thread hiloLetras = new Thread(tareaLetras, "Letras");

        // Momento1: antes de start
        System.out.println("1. Antes de Start(): Numeros = " + hiloNumeros.getState()
                + " Letras = " + hiloLetras.getState());

        // Arrancamos los hilos
        hiloNumeros.start();
        hiloLetras.start();
        // Momento2: despues de start
        System.out.println("2. Después de Start(): Numeros = " + hiloNumeros.getState()
                + " Letras = " + hiloLetras.getState());

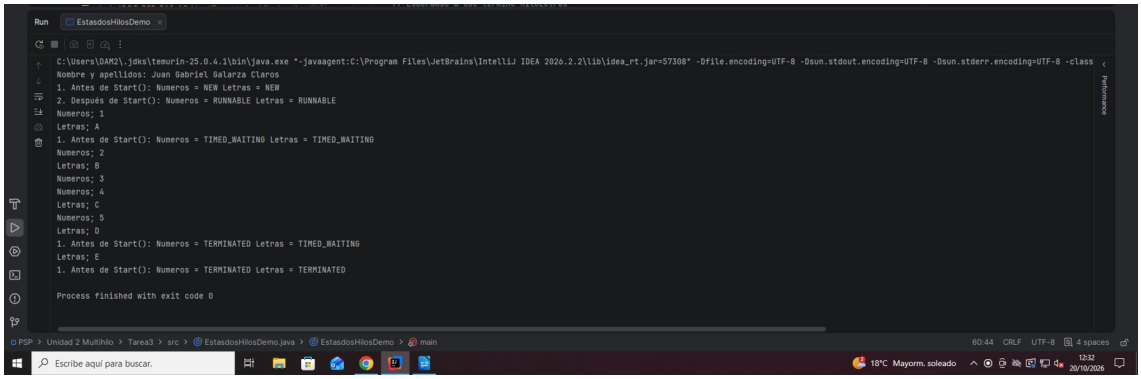
        // Ponemos a dormir el hilo principal (main)
        Thread.sleep(100);
        // Momento3: despues de dormir 100 ms el main
        System.out.println("1. Antes de Start(): Numeros = " + hiloNumeros.getState()
                + " Letras = " + hiloLetras.getState());

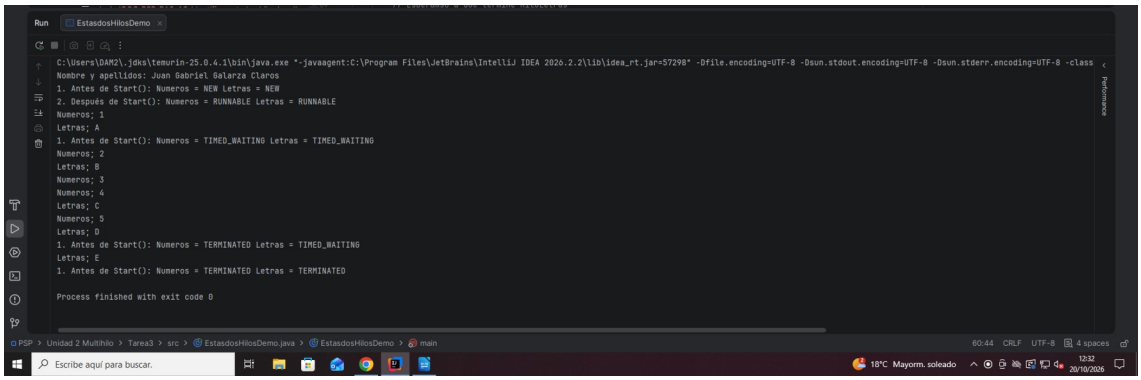
        // Esperamos a que termine hilonumeros
        hiloNumeros.join();
        // Momento4: justo despues que el main espera al hiloNumeros
        System.out.println("1. Antes de Start(): Numeros = " + hiloNumeros.getState()
                + " Letras = " + hiloLetras.getState());

        // Esperamso a que termine hiloLetras
        hiloLetras.join();
        // Momento5: justo despues de que el main espera al hiloLetras
        System.out.println("1. Antes de Start(): Numeros = " + hiloNumeros.getState()
                + " Letras = " + hiloLetras.getState());

    }
}
```

ii. Incluye aquí 2 capturas de pantalla de dos ejecuciones distintas. En ellas deben verse los 5 momentos y la línea con tu nombre.




Run
EstasdosHilosDemo
C:\Users\DAM2\.jdks\temurin-25.0.4.1\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.2\lib\idea_rt.jar=57298" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -class
Nombre y apellidos: Juan Gabriel Galarza Claros
1. Antes de Start(): Numeros = NEW Letras = NEW
2. Después de Start(): Numeros = RUNNABLE Letras = RUNNABLE
Numeros; 1
Letras; A
1. Antes de Start(): Numeros = TIMED_WAITING Letras = TIMED_WAITING
Numeros; 2
Letras; B
Numeros; 3
Numeros; 4
Letras; C
Numeros; 5
Letras; D
1. Antes de Start(): Numeros = TERMINATED Letras = TIMED_WAITING
Letras; E
1. Antes de Start(): Numeros = TERMINATED Letras = TERMINATED
Process finished with exit code 0
Performance
PSP > Unidad 2 Multihilo > Tarea3 > src > EstasdosHilosDemo.java > EstasdosHilosDemo > main
60:44 CRLF UTF-8 4 spaces
Escribe aquí para buscar.
18°C Mayorm. soleado
12:32
20/10/2026